RedHat OpenShift - Développement niveau 1 - Applications de conteneurisation

Utopios® Tous droits réservés - lot 4-002 20/03/2026

Programme de la formation

Jour	Thème	Durée
Jour 1	Introduction et configuration de l'environnement	6h
Jour 2	Développement et déploiement d'applications	6h
Jour 3	Administration avancée et migration	6h

“ **Objectif** : Concevoir, déployer et gérer des applications conteneurisées sur un cluster OpenShift ”

Utopios® Tous droits réservés - lot 4-002 20/03/2026

Objectifs de la formation

- Maîtriser les bases de la conteneurisation avec OpenShift
- Concevoir, construire et déployer des applications conteneurisées
- Gérer les ressources et services associés aux applications
- Migrer et optimiser des applications existantes
- Publier des images dans un registre d'entreprise
- Construire des applications avec Source-to-Image
- Extraire des microservices d'une application monolithique

Utopios® Tous droits réservés - lot 4-002 20/03/2026

Public visé et prérequis

Public visé :

- Développeurs applicatifs
- Administrateurs DevOps
- Ingénieurs en infrastructures cloud et Kubernetes

Prérequis :

- Connaissances de base en Linux et administration système
- Familiarité avec les concepts de conteneurs Docker
- Notions de base sur Kubernetes (facultatif)

Module 1

Introduction à OpenShift et à la conteneurisation

Durée : 1 heure

Qu'est-ce qu'un conteneur ?

Un conteneur est une unité logicielle standardisée qui empaquète le code et toutes ses dépendances.

Caractéristiques clés :

- **Isolation** : processus isolés du système hôte
- **Portabilité** : fonctionne de manière identique partout
- **Légèreté** : partage le noyau du système hôte
- **Rapidité** : démarrage en quelques secondes

> “ Contrairement aux machines virtuelles, les conteneurs ne nécessitent pas leur propre OS complet. ”

Conteneurs vs Machines Virtuelles

Aspect	Conteneur	Machine Virtuelle
Démarrage	Secondes	Minutes
Taille	Mo	Go
Isolation	Processus	Matériel
OS	Partagé (noyau hôte)	Complet par VM
Performances	Quasi-natives	Overhead hyperviseur
Densité	Centaines par hôte	Dizaines par hôte
Orchestration	Kubernetes/OpenShift	vSphere, Hyper-V

Utopios® Tous droits réservés - lot 4-002 20/03/2026

Architecture des conteneurs

```
┌──────────────────────────────────────────────┐
│                Application Layer             │
├──────────────┬──────────────┬────────────────┤
│ Container    │ Container    │   Container    │
│   App A      │   App B      │     App C      │
├──────────────┴──────────────┴────────────────┤
│            Container Runtime                 │
│          (CRI-O / containerd)                │
├──────────────────────────────────────────────┤
│            Linux Kernel (partagé)            │
├──────────────────────────────────────────────┤
│            Infrastructure (physique/VM)      │
└──────────────────────────────────────────────┘
```

Qu'est-ce que Kubernetes ?

Kubernetes (K8s) est un orchestrateur de conteneurs open source :

- **Scheduling** : placement intelligent des conteneurs sur les nœuds
- **Self-healing** : redémarrage automatique des conteneurs défaillants
- **Scaling** : mise à l'échelle horizontale automatique
- **Service Discovery** : découverte et équilibrage de charge
- **Rolling Updates** : mises à jour sans interruption
- **Secret Management** : gestion sécurisée des données sensibles

Qu'est-ce qu'OpenShift ?

Red Hat OpenShift est une plateforme Kubernetes d'entreprise :

- Basée sur **Kubernetes** avec des fonctionnalités supplémentaires
- **Console Web** intégrée pour la gestion visuelle
- **Source-to-Image (S2I)** : construction automatique d'images
- **Routes** : exposition simplifiée des services
- **RBAC** avancé et intégration LDAP/AD
- **Operator Framework** : gestion du cycle de vie des applications
- **CI/CD intégré** : pipelines Tekton natifs
- **Registry interne** : registre d'images intégré

Utopios® Tous droits réservés - lot 4-002 20/03/2026

OpenShift vs Kubernetes vanille

Fonctionnalité	Kubernetes	OpenShift
Console Web	Dashboard basique	Console complète
Sécurité	RBAC de base	SCC, RBAC avancé
Build	Manuel	S2I intégré
Routing	Ingress	Routes natives
Registry	Externe requis	Intégré
CI/CD	Externe	Tekton intégré
Support	Communauté	Red Hat Enterprise
Monitoring	À configurer	Prometheus/Grafana intégré
Opérateurs	OLM optionnel	OperatorHub natif

Utopios® Tous droits réservés - lot 4-002 20/03/2026

Architecture d'OpenShift

```
┌──────────────────────────────────────────────────────────┐
│                        Console Web                       │
├──────────────────────────────────────────────────────────┤
│  Routes | S2I | Registry | Operators | Pipelines         │
├──────────────────────────────────────────────────────────┤
│                   Kubernetes (API Server)                │
├──────────────────────────────────────────────────────────┤
│  Control Plane          │        Worker Nodes            │
│                         │                                │
│  ┌──────────────┐       │  ┌──────────┐  ┌──────────┐   │
│  │ API Server   │       │  │  Pod A   │  │  Pod B   │   │
│  │ etcd         │       │  │  ┌────┐  │  │  ┌────┐  │   │
│  │ Scheduler    │       │  │  │ App│  │  │  │ App│  │   │
│  │ Controller   │       │  │  └────┘  │  │  └────┘  │   │
│  └──────────────┘       │  └──────────┘  └──────────┘   │
├──────────────────────────────────────────────────────────┤
│                  Red Hat CoreOS (RHCOS)                  │
```

Concepts clés d'OpenShift

Projet (Project) : espace de noms isolé avec RBAC

```
oc new-project mon-projet
```

Pod : plus petite unité déployable (1+ conteneurs)
Service : point d'accès réseau stable vers les Pods
Route : expose un Service à l'extérieur du cluster
DeploymentConfig / Deployment : gère le cycle de vie des Pods
BuildConfig : définit comment construire une image
ImageStream : référence et suivi d'images de conteneurs

Utopios® Tous droits réservés - lot 4-002 20/03/2026

Versions et distributions OpenShift

Distribution	Usage
OKD	Version communautaire (upstream)
OpenShift Container Platform (OCP)	Version entreprise Red Hat
OpenShift Dedicated	Cluster managé par Red Hat
ROSA	Red Hat OpenShift on AWS
ARO	Azure Red Hat OpenShift
OpenShift Local (CRC)	Cluster local pour développement

“ Pour cette formation, nous utilisons **OpenShift Local (CRC)** ou un cluster partagé. ”

Module 2

Installation et Configuration d'OpenShift

Durée : 2 heures

Outils nécessaires

OpenShift CLI (`oc`) : outil en ligne de commande principal

```
# Téléchargement (Linux)
wget https://mirror.openshift.com/pub/openshift-v4/clients/ocp/stable/openshift-client-linux.tar.gz
tar xvf openshift-client-linux.tar.gz
sudo mv oc kubectl /usr/local/bin/

# Vérification
oc version
```

Autres outils utiles :

- `kubectl` : CLI Kubernetes standard (inclus)
- `podman` ou `docker` : gestion locale de conteneurs
- `git` : gestion de version du code source

Installation d'OpenShift Local (CRC)

CodeReady Containers (CRC) : cluster OpenShift local

```
# 1. Télécharger CRC depuis console.redhat.com
# 2. Configurer les ressources
crc config set cpus 6
crc config set memory 16384
crc config set disk-size 50

# 3. Initialiser et démarrer
crc setup
crc start

# 4. Récupérer les identifiants
crc console --credentials
```

Connexion au cluster

Via la ligne de commande :

```
# Connexion en tant que développeur
oc login -u developer -p developer \
  https://api.crc.testing:6443

# Connexion en tant qu'administrateur
oc login -u kubeadmin -p <mot-de-passe> \
  https://api.crc.testing:6443

# Vérifier la connexion
oc whoami
oc status
oc cluster-info
```

La Console Web OpenShift

La console offre deux perspectives :

Perspective Administrateur :

- Gestion des nœuds et du cluster
- Monitoring et alertes
- Configuration réseau
- Gestion des opérateurs

Perspective Développeur :

- Topologie des applications
- Builds et pipelines

Utopios® Tous droits réservés - lot 4-002 20/03/2026

Commandes `oc` essentielles

```
oc status                    # État du projet courant
oc get nodes                 # Liste des nœuds
oc get pods                  # Liste des pods
oc get all                   # Toutes les ressources
oc projects                  # Lister les projets
oc project mon-projet        # Changer de projet
oc explain pod               # Documentation d'une ressource
oc logs <pod>                # Logs d'un pod
oc describe pod <pod>        # Description détaillée
oc rsh <pod>                 # Shell interactif dans un pod
```

Configuration du client `oc`

```
# Gestion des contextes
oc config get-contexts
oc config use-context <context>

# Auto-complétion (bash)
source <(oc completion bash)
echo 'source <(oc completion bash)' >> ~/.bashrc

# Auto-complétion (zsh)
source <(oc completion zsh)
echo 'source <(oc completion zsh)' >> ~/.zshrc
```

“ Voir : `demos/jour1/demo-01-connexion-cluster.md` ”

Module 3

Gestion des ressources de base dans OpenShift

Durée : 3 heures

Les Projets OpenShift

Un **Projet** est un espace de noms Kubernetes enrichi avec :

- Isolation des ressources
- Quotas et limites
- Contrôle d'accès (RBAC)

Les Projets OpenShift

```
# Créer un projet
oc new-project formation-openshift \
  --display-name="Formation OpenShift" \
  --description="Projet de formation"

# Lister les projets
oc get projects

# Supprimer un projet
oc delete project formation-openshift
```

Gestion des Projets — Bonnes pratiques

Conventions de nommage :

```
<equipe>-<application>-<environnement>
```

Exemples :

- `dev-frontend-staging`
- `ops-monitoring-prod`
- `formation-openshift-lab`

Gestion des Projets — Bonnes pratiques

Quotas de ressources :

```yaml
apiVersion: v1
kind: ResourceQuota
metadata:
  name: quota-formation
spec:
  hard:
    pods: "10"
    requests.cpu: "4"
    requests.memory: 8Gi
    limits.cpu: "8"
    limits.memory: 16Gi
```

Utopios® Tous droits réservés - lot 4-002 20/03/2026

Gestion des utilisateurs

OpenShift supporte plusieurs fournisseurs d'identité :

- **HTPasswd** : fichier local de mots de passe
- **LDAP** : annuaire d'entreprise
- **OAuth** : GitHub, GitLab, Google
- **OpenID Connect** : Keycloak, Azure AD

Gestion des utilisateurs

```
# Créer un fichier HTPasswd
htpasswd -c -B -b users.htpasswd alice motdepasse
htpasswd -B -b users.htpasswd bob motdepasse

# Créer le secret dans OpenShift
oc create secret generic htpasswd-secret \
  --from-file=htpasswd=users.htpasswd \
  -n openshift-config
```

RBAC — Rôles et permissions

Rôles prédéfinis :

Rôle	Description
`cluster-admin`	Accès total au cluster
`admin`	Gestion complète d'un projet
`edit`	Création/modification des ressources
`view`	Lecture seule
`basic-user`	Accès de base

RBAC — Rôles et permissions

```
# Assigner un rôle à un utilisateur
oc adm policy add-role-to-user admin alice -n mon-projet
oc adm policy add-role-to-user edit bob -n mon-projet
oc adm policy add-role-to-user view charlie -n mon-projet

# Vérifier les permissions
oc adm policy who-can get pods -n mon-projet
oc auth can-i create deployments
```

RBAC — Rôles personnalisés

```yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: deployer
  namespace: mon-projet
rules:
  - apiGroups: ["apps", ""]
    resources: ["deployments", "pods", "services"]
    verbs: ["get", "list", "watch", "create", "update", "delete"]
  - apiGroups: ["route.openshift.io"]
    resources: ["routes"]
    verbs: ["get", "list", "create"]
```

RBAC — Rôles personnalisés

```
oc apply -f role-deployer.yaml
oc create rolebinding alice-deployer --role=deployer --user=alice -n mon-projet
```

Security Context Constraints (SCC)

Les **SCC** contrôlent les actions qu'un Pod peut effectuer :

SCC	Description
`restricted`	Par défaut, très sécurisé
`anyuid`	Permet n'importe quel UID
`privileged`	Accès complet (dangereux)
`nonroot`	Interdit l'exécution en root
`hostnetwork`	Accès au réseau de l'hôte

Security Context Constraints (SCC)

```
# Lister les SCC
oc get scc

# Voir les détails d'un SCC
oc describe scc restricted

# Associer un SCC à un service account
oc adm policy add-scc-to-user anyuid \
  -z mon-service-account -n mon-projet
```

LimitRange — Limites par défaut

```yaml
apiVersion: v1
kind: LimitRange
metadata:
  name: limits-formation
spec:
  limits:
    - type: Container
      default:        { cpu: "500m",  memory: "512Mi" }
      defaultRequest: { cpu: "100m",  memory: "128Mi" }
      max:            { cpu: "2",     memory: "2Gi"   }
      min:            { cpu: "50m",   memory: "64Mi"  }
```

LimitRange — Limites par défaut

```
oc apply -f limitrange.yaml -n mon-projet
oc describe limitrange limits-formation
```

NetworkPolicy — Isolation réseau

```yaml
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-frontend-only
  namespace: mon-projet
spec:
  podSelector:
    matchLabels: { app: backend }
  ingress:
    - from:
        - podSelector:
            matchLabels: { app: frontend }
      ports:
        - protocol: TCP
          port: 8080
```